



# Rapport d'analyse - Projet informatique

2A

## Concurrent SNCF

### **Étudiants :**

Adam BENAMER  
Satya KREMER  
Nathan TIATIA  
Laurine LEROY  
Lou-Ann CLEMENT  
Nour NSIRI

**Encadrant pédagogique :**  
KEVIN LEROY

Année universitaire 2026–2027

## Table des matières

|                                 |          |
|---------------------------------|----------|
| <b>Introduction</b>             | <b>3</b> |
| <b>Diagramme d'architecture</b> | <b>3</b> |
| <b>Diagrammes d'activité</b>    | <b>4</b> |
| Administrateur . . . . .        | 5        |
| Client . . . . .                | 6        |
| Collaborateur . . . . .         | 7        |
| <b>Diagramme de Gantt</b>       | <b>8</b> |

## Introduction

### Diagramme d'architecture

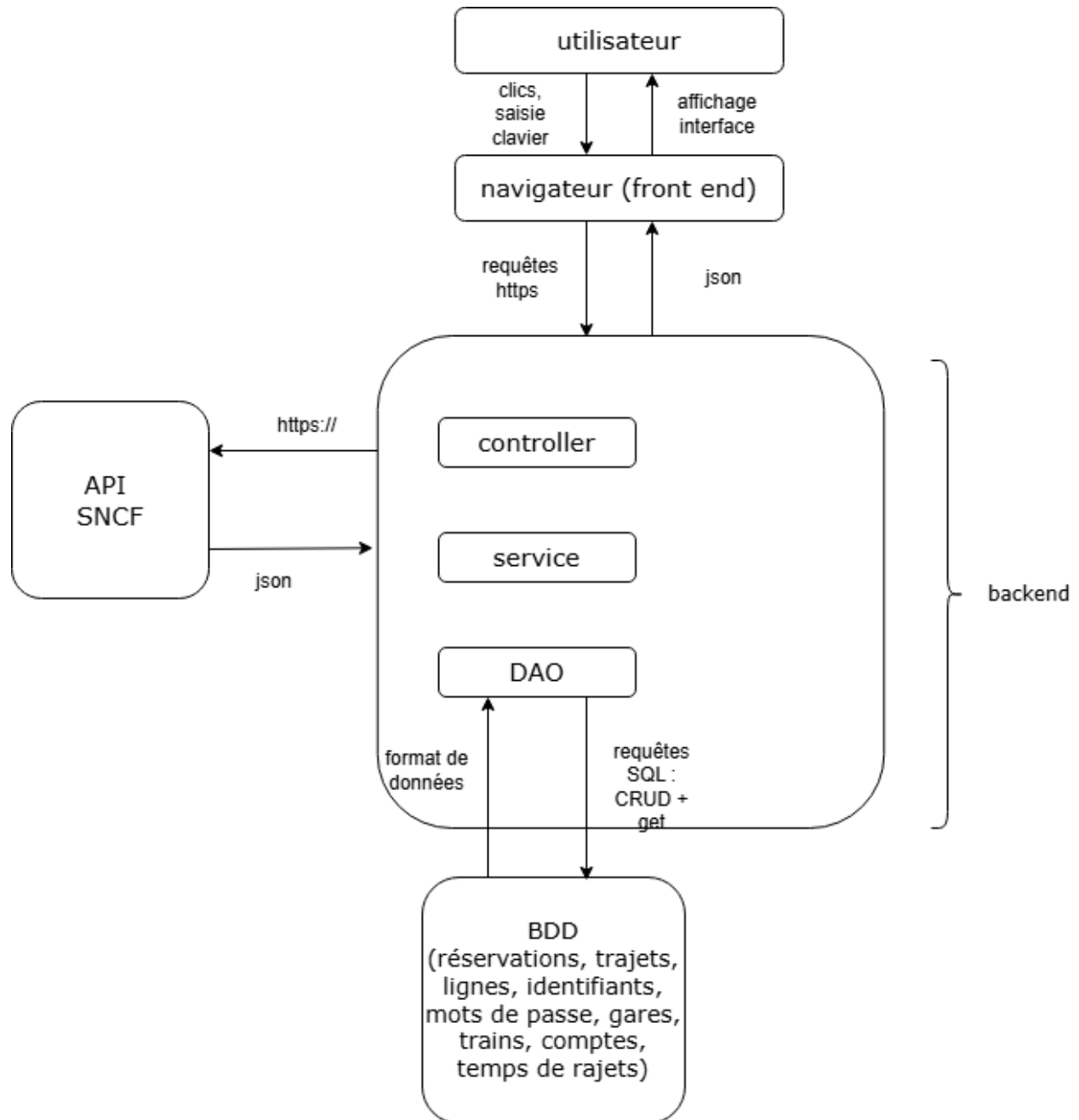


FIGURE 1 : Diagramme d'architecture de l'application

Le diagramme d'architecture ci-dessus illustre la structuration de notre solution logicielle pour notre compagnie ferroviaire. Le système repose sur une architecture client-serveur classique, séparant l'interface utilisateur, le Front-end et la logique métier et de la gestion des données, le Back-end. Au sein du Back-end, nous avons donc :

Le Controller, qui est le point d'entrée de notre API. Il intercepte les requêtes HTTP envoyées par le navigateur, vérifie les habilitations, et retourne les réponses au format JSON.

Le Service, c'est la couche métier. Il contient toute la logique de notre entreprise (vérification de la faisabilité d'un trajet, gestion des planifications, calculs des tarifs). Il orchestre les opérations entre le Controller, l'API SNCF et la base de données. Le DAO, cette couche est exclusivement responsable de la communication avec notre base de données. Elle exécute les requêtes SQL (CRUD : Create, Read, Update, Delete) et transforme les données relationnelles en objets utilisables par notre code.

Par ailleurs, nous avons fait le choix de ne pas interroger l'API SNCF en temps réel à chaque requête de l'utilisateur, mais d'importer et mettre à jour ces données dans notre propre base de données. La principale raison de ce choix est lié à la contrainte de aide à la saisie par autocomplétion lors de la recherche de gare par un utilisateur. Si nous devons faire un appel HTTP vers les serveurs de la SNCF à chaque lettre tapée par l'utilisateur, le temps de latence serait beaucoup trop élevé. Avoir ces données en local dans notre base de données permet des requêtes SQL quasi instantanées, garantissant une autocomplétion fluide.

Ainsi, la mise à jour de notre base de données pourrait s'effectuer :

Soit à l'initialisation : Un script vérifie et met à jour les données manquantes au lancement de l'application. Soit via une tâche planifiée : Le système effectuera une mise à jour ponctuelle en arrière-plan (par exemple, une synchronisation hebdomadaire chaque dimanche à 3h du matin).

## Diagrammes d'activité

Afin de concevoir une API robuste et centrée sur les besoins du métier, nous avons modélisé les processus à travers trois diagrammes d'activité correspondant aux différents niveaux d'habilitation. Les informations en bleu clair correspondent à des fonctionnalités optionnelles et peuvent ne pas être finalement conservées.

## Administrateur

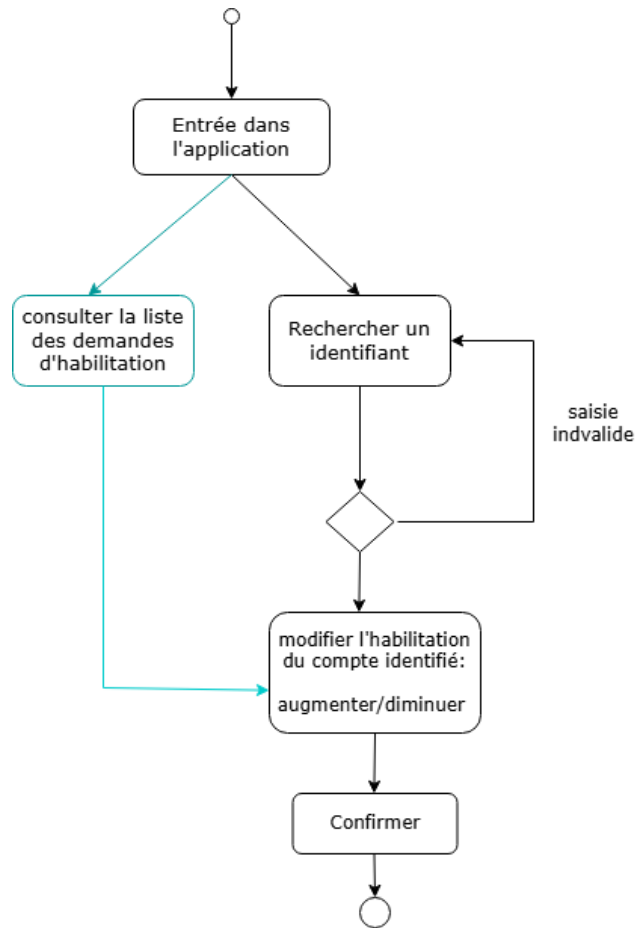


FIGURE 2 : Diagramme d'activité de l'administrateur

Le premier schéma illustre le parcours de l'administrateur, dont le rôle de supervision est restreint et sécurisé. Ce dernier peut rechercher un compte utilisateur spécifique pour consulter son statut actuel, puis augmenter ou diminuer son niveau d'habilitation, par exemple pour promouvoir un client en collaborateur. Le système intègre directement les règles de gestion métier en vérifiant la validité des saisies et en garantissant l'impossibilité de modifier les droits d'un autre administrateur avant de confirmer la transaction en base de données.

## Client

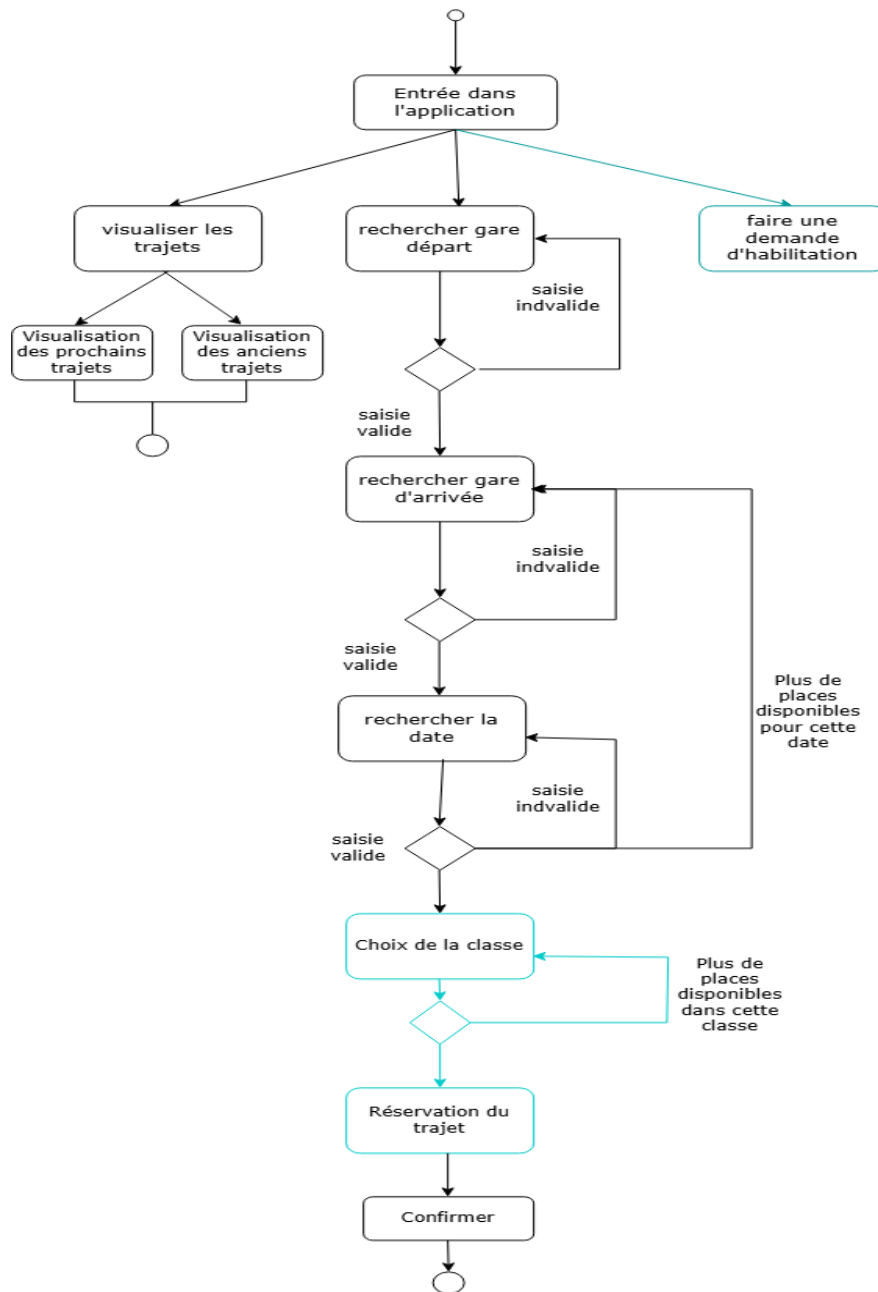


FIGURE 3 : Diagramme d'activité du client

Le deuxième schéma modélise l'expérience utilisateur finale avec le parcours du client, conçu pour être fluide et intégrer des fonctionnalités optionnelles. Le client entame un tunnel de recherche en saisissant ses critères de gares et de dates, avec des validations en temps réel qui lui permettent de modifier facilement sa requête si un trajet s'avère complet sans devoir tout recommencer. Une fois le trajet trouvé, il accède à la réservation où il peut choisir entre une classe classique ou premium selon

les disponibilités. Le client a également la capacité de consulter l'historique de ses réservations au début de l'interface.

## Collaborateur

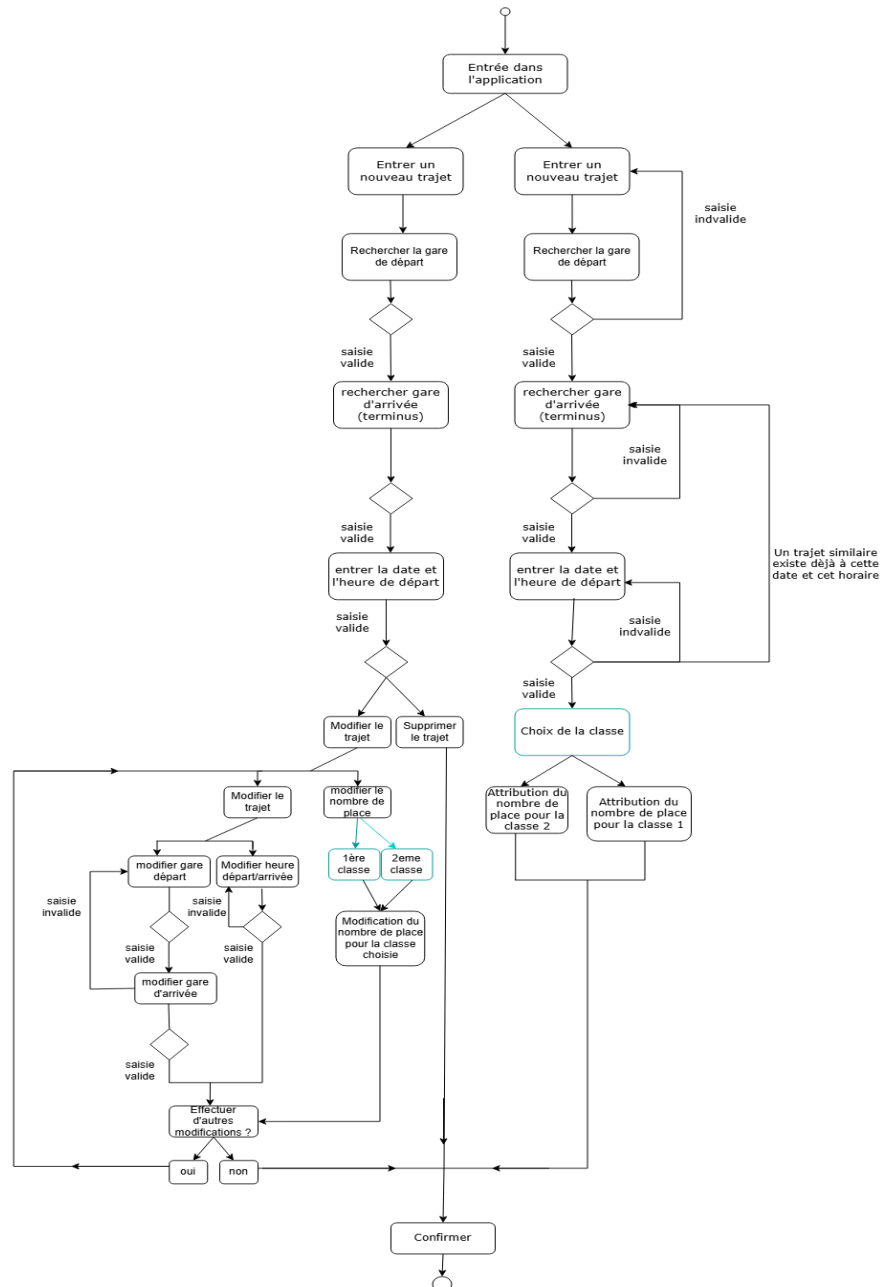


FIGURE 4 : Diagramme d'activité du collaborateur

Enfin, le dernier diagramme détaille le cœur de l'activité d'exploitation ferroviaire à travers le parcours du collaborateur, divisé en deux axes de gestion. D'une part, le collaborateur peut créer une nouvelle ligne en indiquant la gare de départ et d'arrivée ainsi que la date et l'heure de cette nouvelle ligne. Il alloue également un nombre de places à chacune de ses classes. D'autre part, il a la possibilité de chercher un trajet existant afin de modifier (l'horaire, la gare de départ ou d'arrivée, le nombre de places disponible pour chaque classe) ou de supprimer une ligne.

## Diagramme de Gantt

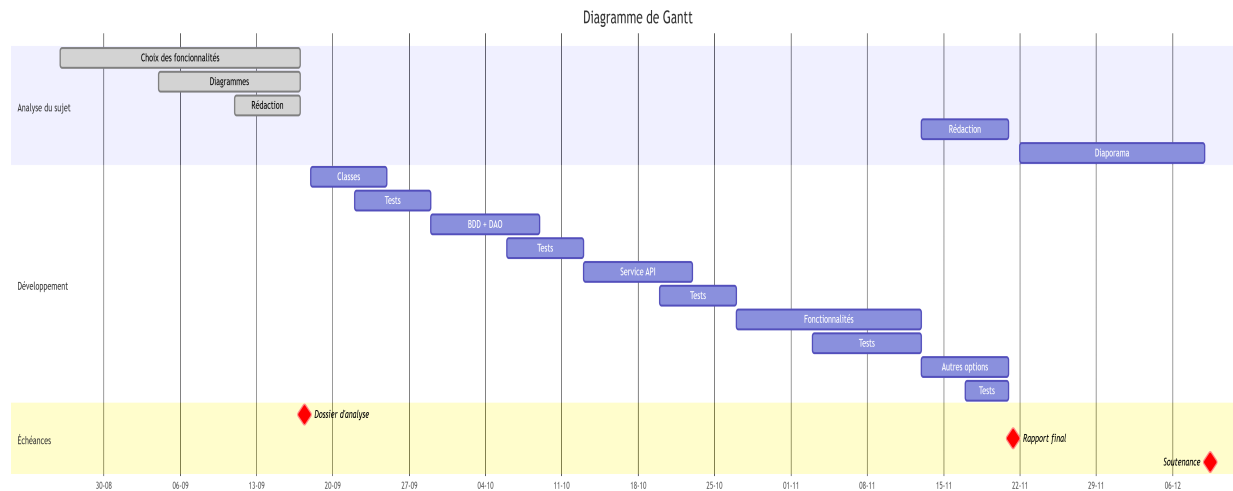


FIGURE 5 : Diagramme de Gantt